

# Task 2 - User Authentication System (Flask)

Extends Task 1 with **registration, login, sessions, and a protected dashboard**, built with Python, Flask, SQLite and Werkzeug password hashing.

## Features

- User registration with a unique username
- Passwords are hashed with Werkzeug before being stored (never saved as plain text)
- Login checks the hash, not the raw password
- Session-based authentication (Flask's built-in session)
- /dashboard is a protected route: only logged-in users can view it
- Logout clears the session

## Technologies Used

| Layer    | Technology                                |
|----------|-------------------------------------------|
| Backend  | Python 3.10+, Flask                       |
| Frontend | HTML, CSS (Jinja2 templates)              |
| Database | SQLite                                    |
| Security | Werkzeug password hashing, Flask sessions |

## Project Structure

```
python-fullstack-task2/
├── app.py                # Flask server: register, login, dashboard, logout
├── database.db           # SQLite database (users table)
├── requirements.txt
├── templates/
│   ├── register.html
│   ├── login.html
│   └── dashboard.html
├── static/
│   └── style.css
```

## Authentication Flow

- **Register** (/register): user submits a username and password. The password is hashed with `generate_password_hash()` and stored in the users table. If the username already exists, an error is shown instead of crashing.
- **Login** (/login): user submits username and password. The app looks up the username, then checks the password against the stored hash with `check_password_hash()`. On success, `session['user']` is set to the username.
- **Session**: Flask signs the session cookie with `app.secret_key`, so the browser stays "logged in" across requests without the server storing session state in the database.
- **Protected route** (/dashboard): before rendering, the route checks if 'user' not in session and redirects to /login if the visitor isn't authenticated. This is what "access control" means in the task brief.
- **Logout** (/logout): removes user from the session, so the next visit to /dashboard redirects to login again.

## Database Schema

```
CREATE TABLE users (  
    id INTEGER PRIMARY KEY,  
    username TEXT UNIQUE,  
    password TEXT  
);
```

password stores a hash (e.g. pbkdf2:sha256: . . .), never the plain text.

## How to Run

```
pip install flask werkzeug  
python app.py
```

Then open **http://127.0.0.1:5000/register**, create an account, then log in at

**http://127.0.0.1:5000/login**.

## Screenshots

Add to a screenshots/ folder and link here:

```
! [Register](screenshots/register.png)  
! [Login](screenshots/login.png)  
! [Dashboard](screenshots/dashboard.png)
```

## Security Notes

- `app.secret_key` is a placeholder here — for anything beyond a class task, use a long random value and load it from an environment variable, not hardcoded in the file.
- Passwords are never stored or logged in plain text.

## Author

Your Name